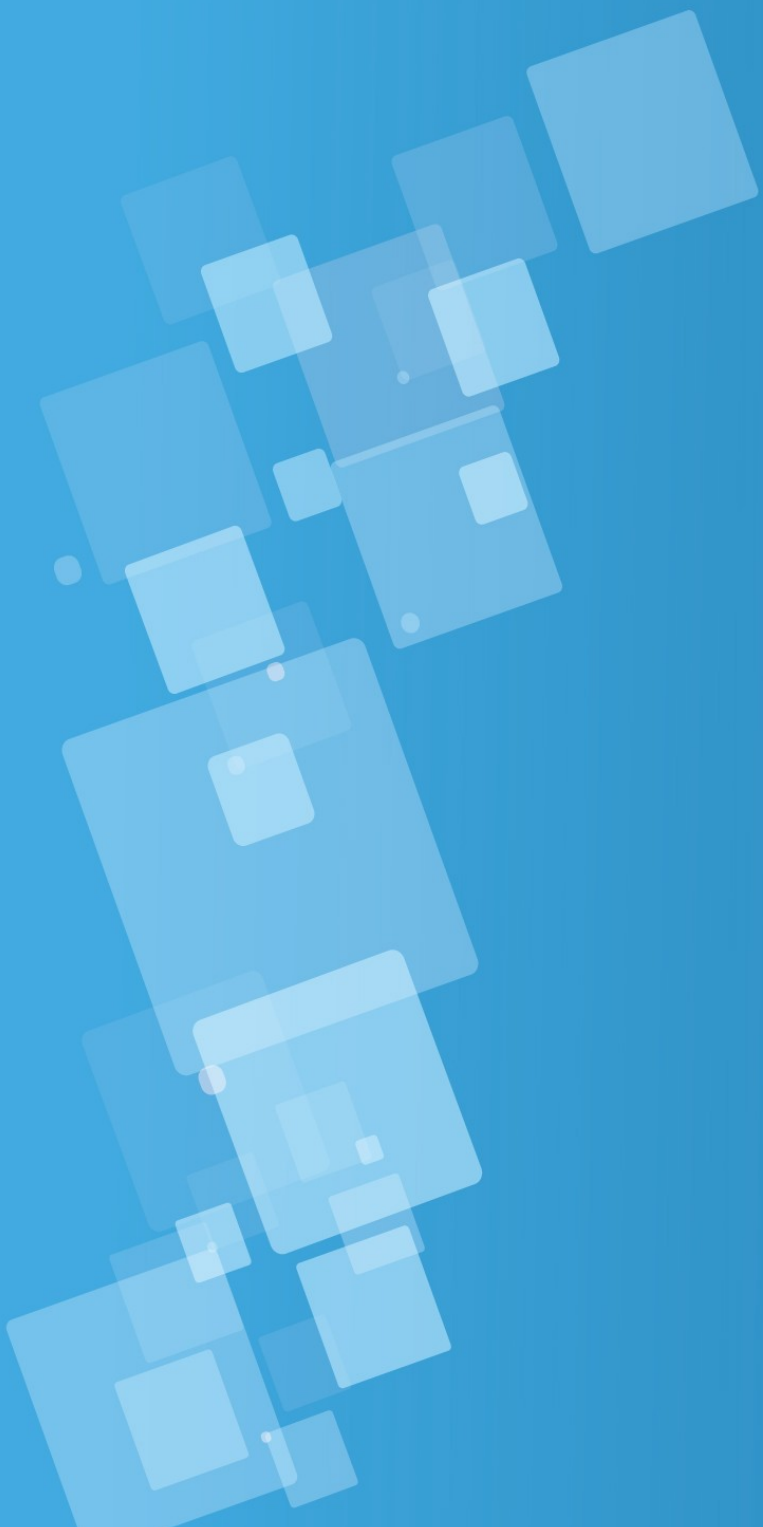




Técnicas Digitales II

Microprocesadores ARM

- Bibliografía

Computer Organization and Design ARM Edition, 2017
Capítulo 4.9

The Definitive Guide to ARM Cortex-M3 and Cortex-M4
Processors 3ra Edición de Joseph Yiu
Capitulo 6

- Soporte

Harris & Harris. Digital design and computer architecture:
ARM edition. Elsevier, 2015. Capítulo 6.



Arquitectura

Define el funcionamiento de un microprocesador, es el diseño conceptual y está compuesta por:

- El set de instrucciones.
- Registros
- Modelo de excepciones.
- La ubicación de los operandos con respecto a la CPU y de que manera esta trabaja internamente.

Ejemplo de algunas arquitecturas:
ARM, x86 (Intel/AMD), MIPS, SPARC, PowerPC



Arquitectura

- Cualquier soft que deba correr en un microprocesador es compilado en una serie de instrucciones simples como (sumas, restas y saltos).
- Estas instrucciones poseen la operación y los operandos que utilizará (registros, memoria o un valor en la misma operación).
- Como una computadora entiende solo 1 y 0, estas instrucciones están codificadas en binario en un formato denominado Código Máquina. (ARM por ejemplo cada instrucción tiene 32 bits).
- Para que su lectura sea menos tediosa, estas instrucciones se representan en un formato simbólico con el uso de mnemónicos, este nuevo formato se llama Lenguaje Ensamblador.



Arquitectura

- Una arquitectura no define la implementación en hardware, en general existe varias implementaciones para una misma arquitectura.

Ejemplo: Intel y Advanced Micro Devices (AMD) fabrican microprocesadores con arquitectura x86

- Sus implementaciones se realizan con distinto diseño de hardware, poseen diferentes precios y rendimientos.
- Se optimizan con diferentes objetivos, por ejemplo mejora de rendimiento en servidores, economizar energía, etc.

Misma arquitectura significa igual set de instrucciones, pueden correr el mismo software.



Arquitectura RISC vs CISC

- CISC (Complex Instruction Set Computer) o Computador con Conjunto de Instrucciones Complejas.
- RISC (Reduced Instruction Set Computer) o Computador con Conjunto de Instrucciones Reducidas.



Arquitectura RISC vs CISC

- CISC (Complex Instruction Set Computer) o Computador con Conjunto de Instrucciones Complejas
 - Esta arquitectura tiene un set de instrucciones mas amplio.
 - Permite operaciones entre memoria o memoria/registro.
 - Motorola 68000, Zilog Z80 y toda la familia Intel x86, AMD x86-64.



Arquitectura RISC vs CISC

- RISC (Reduced Instruction Set Computer) o Computador con Conjunto de Instrucciones Reducidas
 - Set de instrucciones pequeño y de tamaño fijo , esto resulta en hardware para decodificar la instrucción mas simple, pequeño y rápido.
 - Solo existen instrucciones de carga y almacenamiento para acceder a memoria.
 - Suelen tener mayor cantidad de registros.
 - PowerPC, DEC Alpha, MIPS, ARM, SPARC



Arquitectura de conjunto de instrucciones (ISA)

- Conjunto de instrucciones que realiza el microprocesador.
- Formada en general por operaciones básicas como sumas y restas.
- Cada instrucción indica la operación y los operandos a usar.
- Los operandos pueden ser registros, memorias o un valor inserto en la misma operación.
- Las instrucciones son codificadas como un número binario en un formato denominado código máquina.
- El ARM representa a cada instrucción con un número de 32 o 16 bits.



ARM

En los capítulos siguientes se desarrollará la arquitectura ARM

- Desarrollada por Acorn Computer Group en la década de 1980.
- Acrom junto con Apple crean en 1990 Advanced RISC Machine (ARM).
- La arquitectura ARM es licenciable. El negocio principal de ARM Holdings es la venta de núcleos.
- Es una de las arquitecturas mas difundidas (Wikipedia)
 - 98% de los mil millones de celulares vendidos anualmente.
 - 90% de microprocesadores RISC.



ARM

- Empresas que poseen licencia ARM: (Wikipedia)

Alcatel-Lucent, Apple Inc., AppliedMicro, Atmel, Broadcom, Cirrus Logic, Digital Equipment Corporation, Ember, Energy Micro, Freescale, Intel (a través de DEC), LG, Marvell Technology Group, Microsemi, Microsoft, NEC, Nintendo, Nokia, Nuvoton, Nvidia, Sony, MediaTek, NXP (antes Philips Semiconductors), Oki, ON Semiconductor, Psion, Qualcomm, Samsung, Sharp, STMicroelectronics, Symbios Logic, Texas Instruments, VLSI Technology, Yamaha, y ZiiLABS.



ARM

- SO embebidos y RT que soportan ARM: (Wikipedia)

Android, Windows CE, Windows 8 RT, .NET Micro Framework, Symbian, ChibiOS/RT, FreeRTOS, eCos, Integrity, Nucleus PLUS, MicroC/OS-II, QNX, RTEMS, BRTOS, RTX C Quadros, ThreadX, Unison Operating System, uTasker, VxWorks, MQX y OSE

- SO

Solaris, BSD, Linux, Plan9, Apple iOS, Windows



ARM Las Versiones de su Arquitectura

- ARMv4 (1994) ARM7TDMI, StrongARM: Se masificó la arquitectura
 - Característica: Introducción del set Thumb (instrucciones de 16 bits).
 - Ejemplos: La Game Boy Advance o el Nokia 1100. Usaban Thumb para meter juegos complejos en cartuchos o memorias de apenas unos pocos megabytes.
- ARMv5 (2000) ARM9, ARM10 : El inicio del multimedia
 - Característica: Instrucciones DSP (Digital Signal Processing) para aritmética saturada.
 - Ejemplos: Los primeros iPod y reproductores MP3. Estas instrucciones permitían decodificar audio en tiempo real sin necesidad de un chip dedicado muy caro.



ARM Las Versiones de su Arquitectura

- ARMv6 (2002) ARM11 (Raspberry Pi 1): Paralelismo básico
 - Característica: Instrucciones SIMD (Single Instruction Multiple Data) de bajo nivel.
 - Ejemplo: se podía procesar colores de píxeles en paralelo, lo que permitía interfaces gráficas fluidas en Linux.
- ARMv6-M (2007) Cortex-M0, M1: El reemplazo del 8-bits
 - Característica: Set de instrucciones minimalista (casi exclusivamente 16 bits).
 - Ejemplo práctico: Los Cortex-M0 y M0+ de muy bajo costo, reemplaza a los viejos PIC16 por ser más fácil de programar en C y consumir menos energía.



ARM Las Versiones de su Arquitectura

- ARMv7-M (2004) Cortex-M3 y M4: El estándar actual de microcontroladores
 - Característica: Tecnología Thumb-2 y el NVIC (controlador de interrupciones).
 - Ejemplo: Aplicaciones mas compleja con interrupciones por mejorar el determinismo y disminuir latencias.
- ARMv7-A (2005) Cortex-A8, A9, A15 : La explosión de los Smartphones
 - Característica: Motor NEON (procesamiento vectorial de 128 bits).
 - Ejemplo: La grabación de video HD en un iPhone 4 o Samsung Galaxy S.



ARM Las Versiones de su Arquitectura

- ARMv8-A (2011) Cortex-A53, A72, Apple A7: El salto a los 64 bits
 - Característica: Arquitectura AArch64 (registros de 64 bits y direccionamiento masivo).
 - Ejemplo: Servidores de Amazon (Graviton) o el iPhone 5s 1er smart de 64 bits. Permite gestionar más de 4GB de RAM y procesar archivos de video 4K/8K con precisión de 64 bits.
- ARMv8-M (2015) Cortex-M23, M33: Seguridad para el IoT
 - Característica: TrustZone para microcontroladores.
 - Ejemplo práctico: Una cerradura inteligente Wi-Fi. TrustZone separa el código de la conexión Wi-Fi (propenso a ataques) de la llave digital (protegida en la zona segura del hardware).



ARM Las Versiones de su Arquitectura

- ARMv9 (2021) Cortex-X2, A710: Inteligencia Artificial nativa
 - Característica: Extensiones SVE2 (Scalable Vector Extension).
 - Ejemplo: Reconocimiento facial instantáneo en el celular. SVE2 permite que el procesador haga trillones de operaciones matemáticas de IA por segundo sin calentar el equipo.



ARM Las Versiones de su Arquitectura

Versión	Año Ap.	Características Clave	Núcleos Famosos	Hito Tecnológico
ARMv4	1994	Introducción del set Thumb (16 bits)	ARM7TDMI, StrongARM	Nokia 1100, Apple Newton
ARMv5	2000	Instrucciones de DSP y mejoras C.	ARM9, ARM10	Mejor rendimiento en audio y video básico.
ARMv6	2002	Instrucciones SIMD básicas y manejo de memoria mejorado.	ARM11 (Raspberry Pi 1)	Primera generación de smartphones modernos.
ARMv6-M	2007	Subconjunto minimalista para bajo consumo.	Cortex-M0, M1	El reemplazo natural de los 8 bits (PIC/AVR).
ARMv7-A	2005	Introducción de NEON (SIMD avanzado) y multinúcleo.	Cortex-A8, A9, A15	El boom de Android y el iPhone original.
ARMv7-M	2004	Thumb-2 (mezcla 16/32 bits), NVIC integrado.	Cortex-M3 y M4	El estándar de la industria microcontroladora actual.
ARMv8-A	2011	El gran salto a 64 bits (AArch64). Reg. de 64 bits.	Cortex-A53, A72, Apple A7	iPhone 5s
ARMv8-M	2015	Seguridad TrustZone para microcontroladores.	Cortex-M23, M33	IoT seguro (Internet de las Cosas).
ARMv9	2021	Enfoque en Inteligencia Artificial y computación confidencial.	Cortex-X2, A710	Optimización para aprendizaje automático y seguridad total.



Navegando en un Mar de Arquitecturas

- Vimos que existen una diversidad de arquitecturas (x86, RISC-V, ARM) y versiones (ARMv4 a ARMv9) que difieren profundamente entre sí.
- Analizar simultáneamente varias arquitecturas, implica un conocimiento superficial de todas, nos enfocamos en una particular: la arquitectura ARMv7-M (Cortex-M3).
- Vamos a analizar esta arquitectura para conocerla en profundidad. La idea es que, una vez dominados estos conceptos, seas capaz de trasladar este conocimiento a cualquier otra arquitectura profesional en el futuro.



La Herramienta de Trabajo: **Blue Pill**

- El Hardware: Placa de desarrollo basada en el microcontrolador STM32F103C8T6.
- Microcontrolador: Un núcleo ARM Cortex-M3 de 32 bits corriendo a 72 MHz.
- ¿Por qué esta placa?:
 - Estándar de la industria para sistemas embebidos de gama media.
 - Posee una relación costo-rendimiento muy buena.
 - Permite un control total del hardware, ideal para aprender qué sucede realmente.

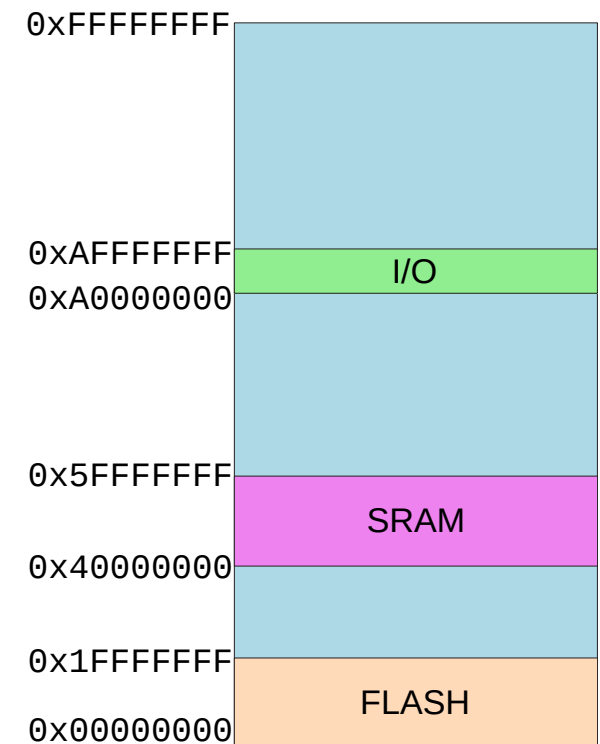
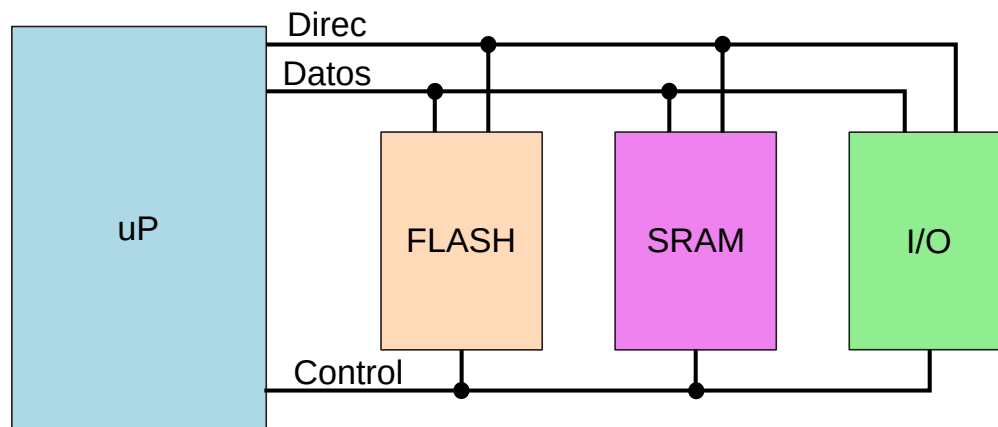


Programación Profesional con Toolchains Abiertos

- El Entorno: Utilizaremos el compilador estándar de la industria: GNU GCC (arm-none-eabi-gcc).
- Sin IDEs propietarios. Controlaremos el proceso de:
 - Compilación: Traducir nuestro C a código objeto.
 - Linkeo: Organizar la memoria mediante Linker Scripts (.ld).
 - Depuración Real: Usaremos GDB y OpenOCD para realizar debug en tiempo real sobre el silicio, inspeccionando registros y memoria paso a paso.

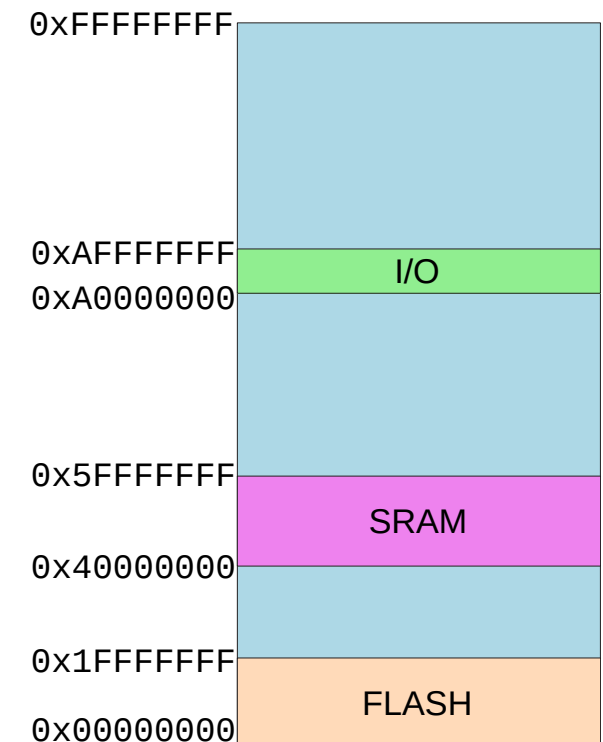
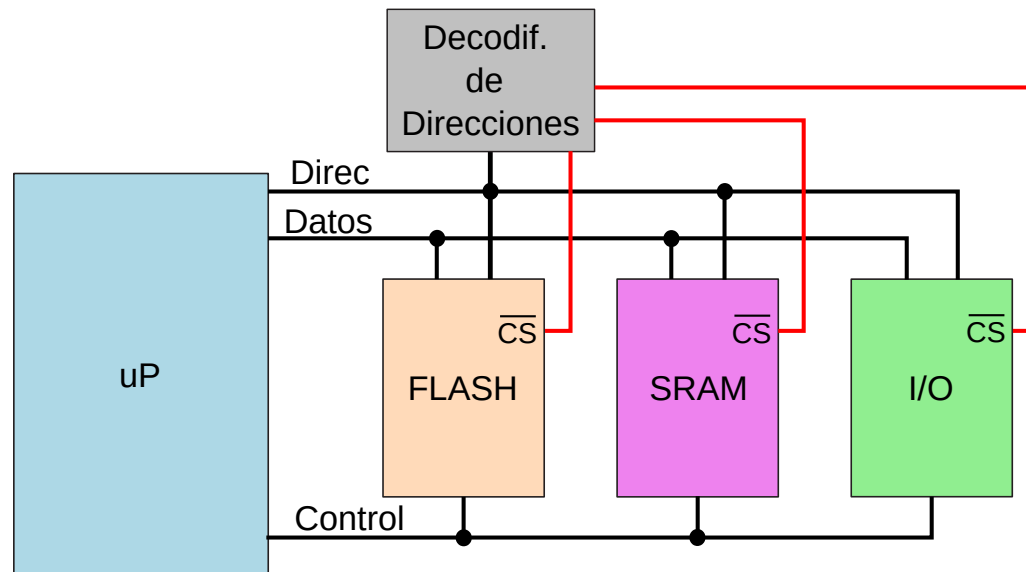
El Decodificador de Direcciones

- En todo sistema, el bus de direcciones del microprocesador, está conectado a todos los dispositivos simultáneamente
 - Memoria SRAM
 - Memoria FLASH
 - Dispositivos de I/O
- ¿Cómo evitamos que todos respondan al mismo tiempo?



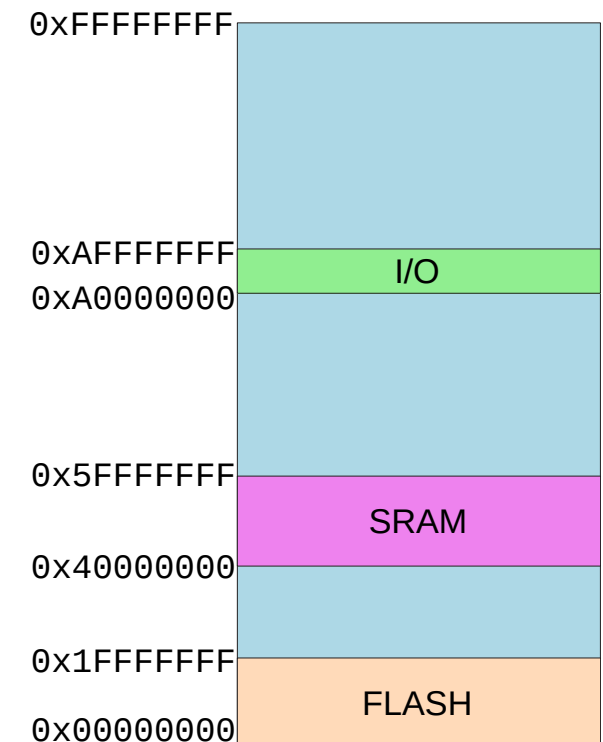
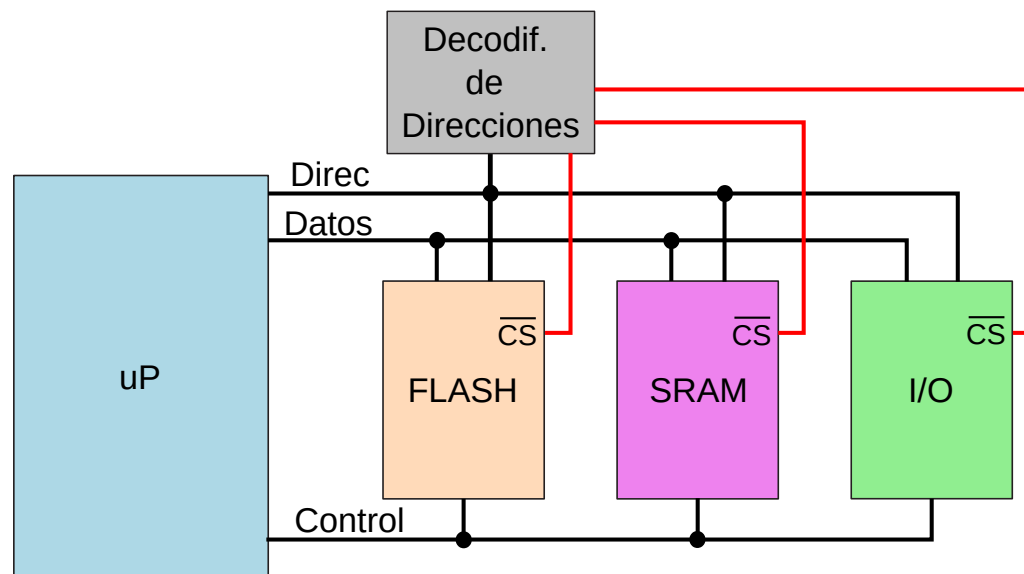
El Decodificador de Direcciones

- **Solución:** Una red de compuertas lógicas llamada Decodificador de Direcciones.
- Esta lógica "escucha" los bits de mayor peso (MSB) del bus de direcciones y genera una señal de activación única llamada Chip Select (CS) o Chip Enable (CE).



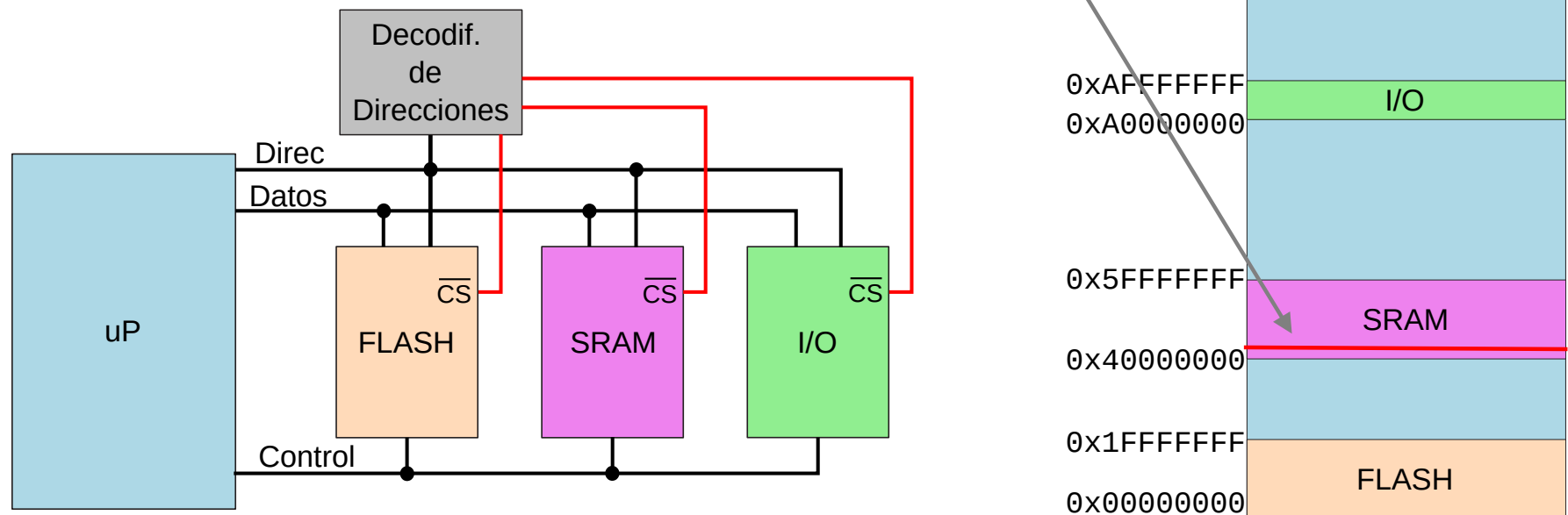
El Decodificador de Direcciones

- El "Mapeo" entonces es un proceso de decodificación de hardware:



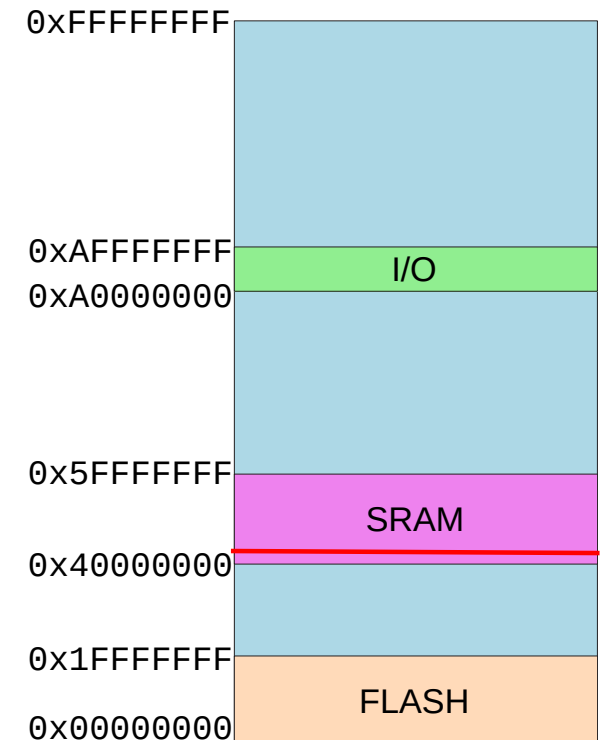
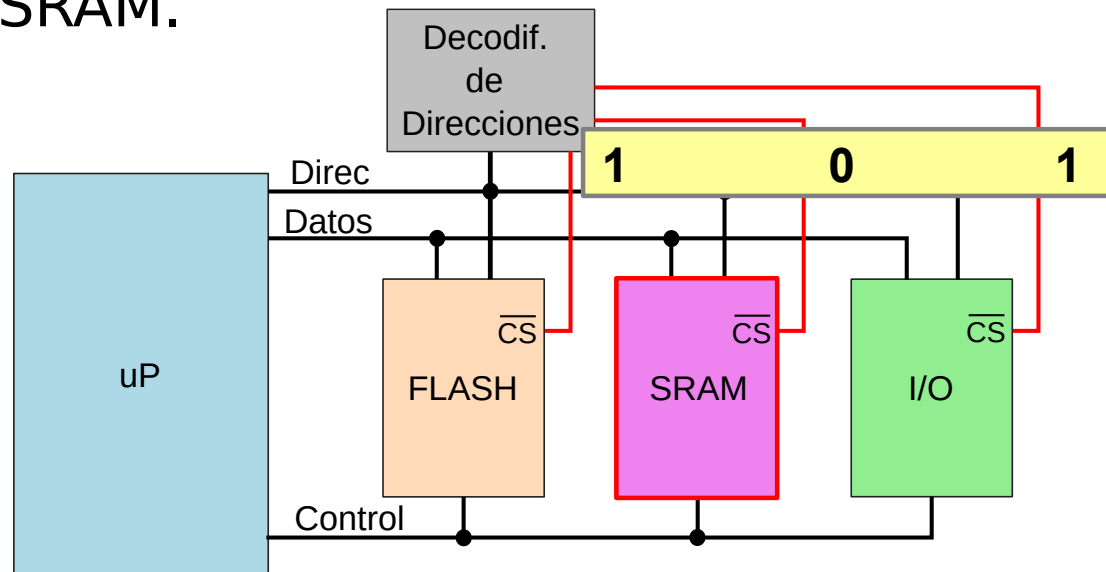
El Decodificador de Direcciones

- El "Mapeo" entonces es un proceso de decodificación de hardware:
- La CPU coloca una dirección en el bus (ej. 0x4001080C).
-



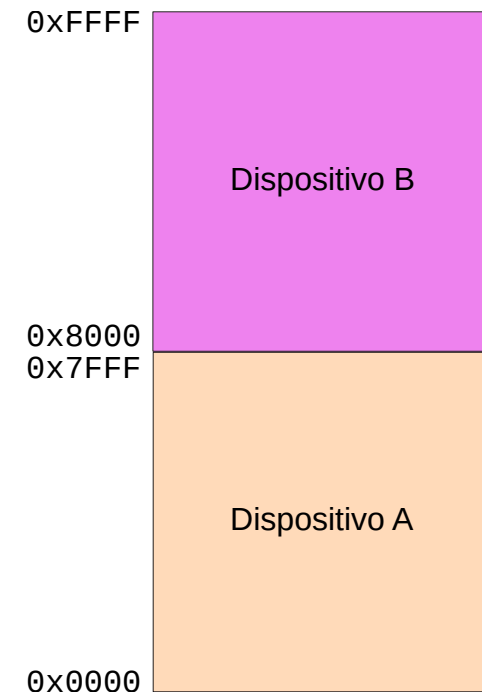
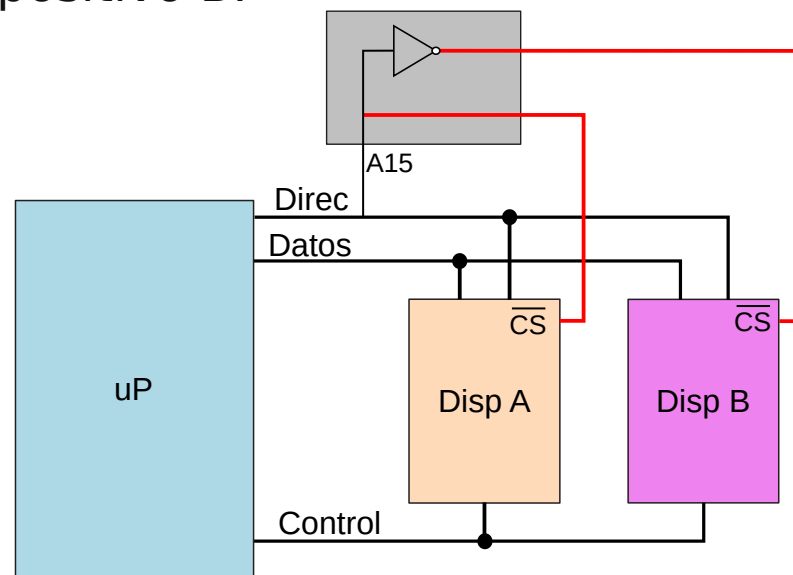
El Decodificador de Direcciones

- El "Mapeo" entonces es un proceso de decodificación de hardware:
- La CPU coloca una dirección en el bus (ej. 0x4001080C).
- El decodificador de instrucciones interpreta los bits de mayor peso para activar el "chip select" del periférico correspondiente, en este caso la SRAM.

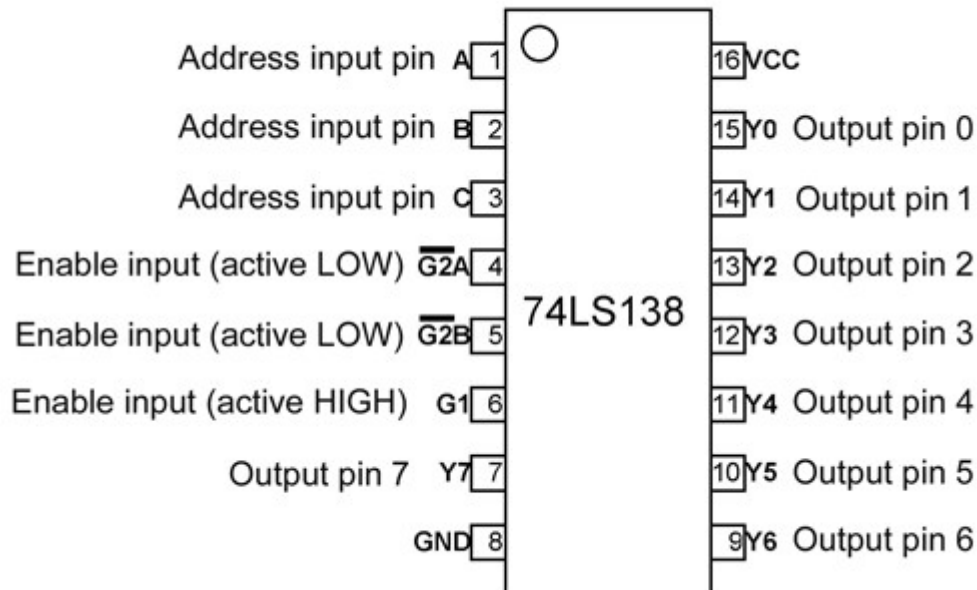


Ejemplo Práctico con Compuertas

- Imaginemos un sistema simple con un bus de 16 bits y dos chips:
 - Dispositivo A: Mapeado en 0x0000 - 0x7FFF (Bit A15=0).
 - Dispositivo B: Mapeado en 0x8000 - 0xFFFF (Bit A15=1).
- La Lógica Física:
 - Conectamos el cable A15 directamente al pin $\overline{CS}$ del Dispositivo A.
 - Conectamos el cable A15 a través de un inversor (NOT) al pin $\overline{CS}$ del Dispositivo B.



- En sistemas digitales clásicos (y dentro del silicio del STM32), se usan bloques como el decodificador 3 a 8: 74LS138
 - Entrada: 3 bits del bus de direcciones.
 - Salida: 8 líneas de CS diferentes.
- | Input | | | | | | Output | | | | | | | |
|--------|-----|----|--------|---|---|--------|---|---|---|---|---|---|--|
| Enable | | | Select | | | | | | | | | | |
| G2A | G2B | G1 | C | B | A | 0 | 1 | 2 | 3 | 4 | 5 | 6 | |

[illegible]



Ejercicio de Diseño: Decodificación de Direcciones

- Diseñar la lógica física para mapear 3 dispositivos en un bus de direcciones de 16 bits (A15 a A0):
 - Dispositivo 1 (EPROM): Comienza de 0x0000 de 16 KB.
 - Dispositivo 2 (RAM): Comienza de 0x4000 de 16 KB.
 - Dispositivo 3 (I/O): Comienza de 0x8000 de 16 KB.
- ¿Qué señales lógicas debemos conectar a los pines CS (Chip Select) de cada integrado para que el micro pueda acceder a ellos sin conflictos?



Entrada / Salida (I/O)

- Las entradas/salidas en un sistema, permite conectar el mismo con el exterior por medio de dispositivos denominados periféricos.
- Muchas veces hablamos de periféricos como Monitores, Teclados o Impresoras.
- Estos son dispositivos o periféricos externos al chip (Off-Chip), los cuales estarán manejados por periféricos internos, de silicio o simplemente dispositivos de I/O, como: GPIO, UART ,I2C, etc
- Estos periféricos internos, son los que estudiaremos

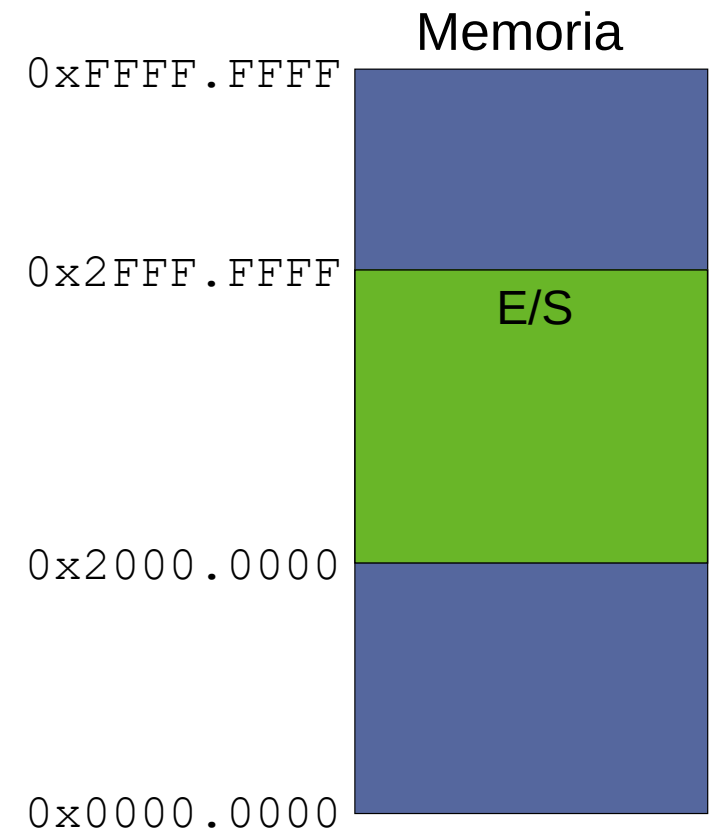


Entrada / Salida (I/O)

- Vimos que en todo sistema, el microprocesador, está conectado a una serie de dispositivos, como ser:
 - Memoria DRAM
 - Memoria FLASH
 - Dispositivos de I/O
- Una porción de su espacio de direcciones está dedicada a comunicarse con los dispositivos de I/O.
- Esta porción de su mapa no accede a memoria, en cambio direcciona dispositivos de I/O.

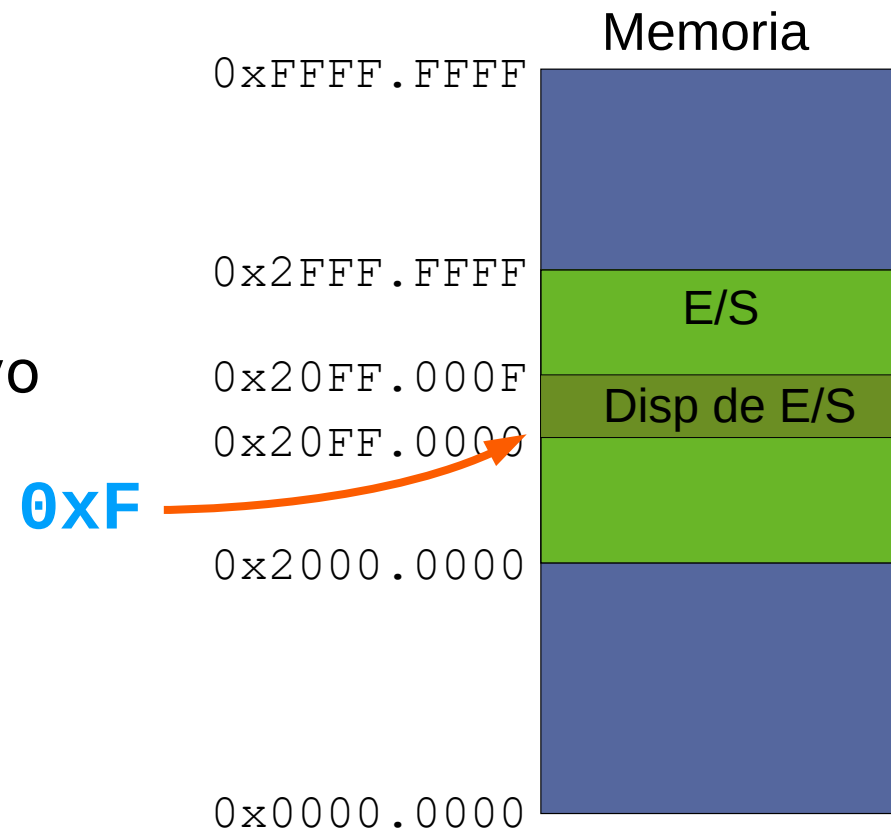
Entrada / Salida (I/O)

- En un ejemplo, un sistema posee el rango de direccionamiento de 0x2000.0000 a 0x2FFF.FFFF asignado a E/S.
- En ese rango se encontrarán todos los dispositivos de E/S del sistema.



Entrada / Salida (I/O)

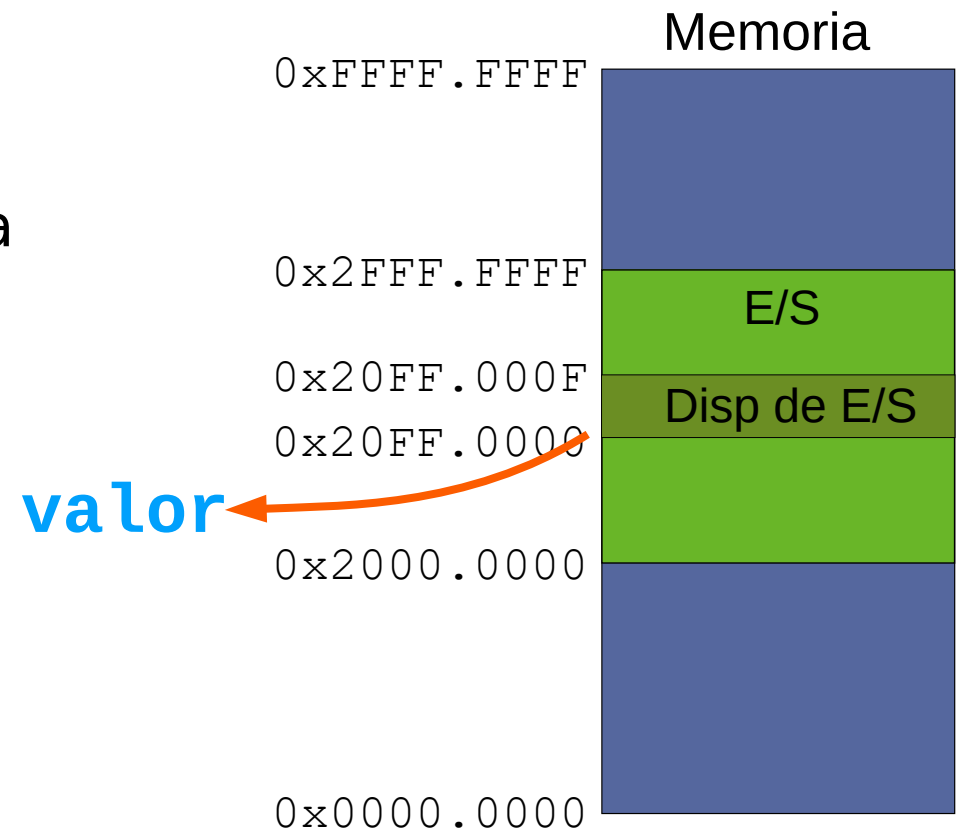
- Cada dispositivo tendrá una o varias direcciones asignadas. (en el ejemplo 16 bytes desde la dirección 0x20FF.0000)
- Una escritura a una dirección de memoria asignada a un dispositivo indica un envío de datos a el dispositivo



```
#define PERIPHERAL_ADDR 0x20FF0000
*(volatile uint32_t *)PERIPHERAL_ADDR = 0xF;
```

Entrada / Salida (I/O)

- En caso inverso, una lectura en una dirección de memoria asignada a un dispositivo indica una lectura de datos desde el dispositivo



```
#define PERIPHERAL_ADDR 0x20FF0000
uint32_t valor;
valor = *(volatile uint32_t *)PERIPHERAL_ADDR;
```

Modelos de Entrada/Salida (I/O)

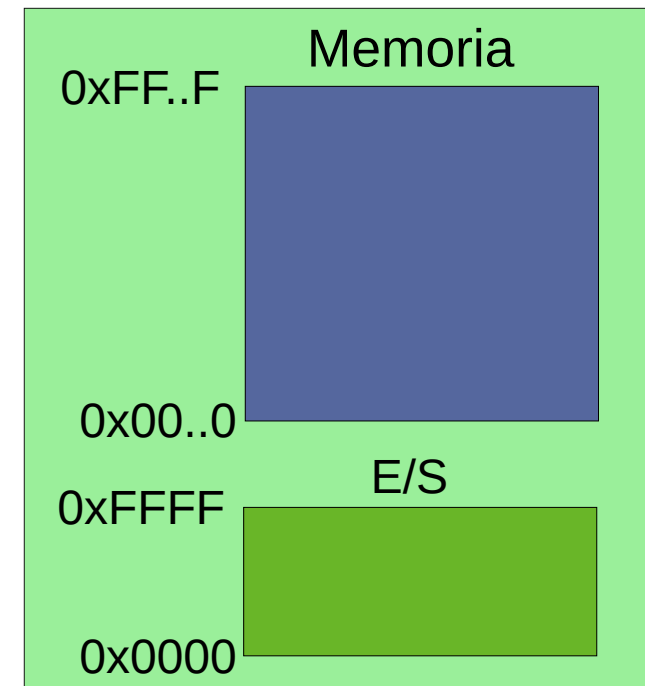
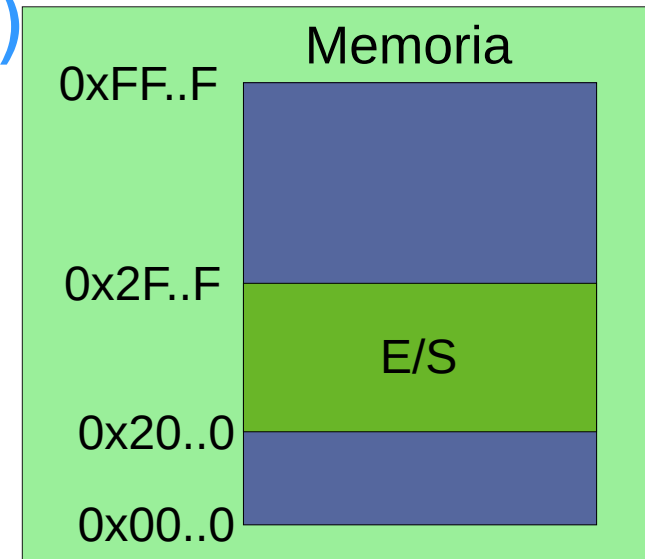
Existen dos filosofías principales en la arquitectura de computadoras para hablar con el hardware:

I/O Mapeada en Memoria (Memory-Mapped I/O)

Los periféricos comparten el mismo espacio de direcciones que la memoria RAM y Flash. No se requieren instrucciones especiales.

I/O Aislada (Port-Mapped I/O)

El procesador tiene instrucciones especiales (IN, OUT) y un espacio de direcciones separado para periféricos. (Ejemplo: Arquitectura x86).





¿Por qué ARM elige Mapeo en Memoria?

Simplicidad y potencia del set de instrucciones:

- **Un solo conjunto de instrucciones:** En ensamblador veremos que solo usamos LDR (Load) para leer un estado y STR (Store) para enviar un comando).
- **Flexibilidad de punteros:** Permite usar todas las capacidades del lenguaje C (punteros, estructuras, desplazamientos) para manipular el hardware.
- **Ortogonalidad:** Todas las formas de direccionamiento que funcionan para la RAM funcionan también para los periféricos (modos indexados, incrementos automáticos).



El Registro como Abstracción de Hardware

- Las direcciones asociadas con los dispositivos de E/S se llaman registros.
- Los registros, son un conjunto de Flip-Flops (latches) conectados físicamente a los pines del chip o a la lógica interna (Timers, ADC).
- Al ejecutar una escritura a una dirección de memoria de un dispositivo o modificar una variable en C que apunta a un dispositivo, estamos físicamente cambiando el estado eléctrico de esos latches, lo que resulta en una acción en el mundo real (ej. prender un motor).

BluePill

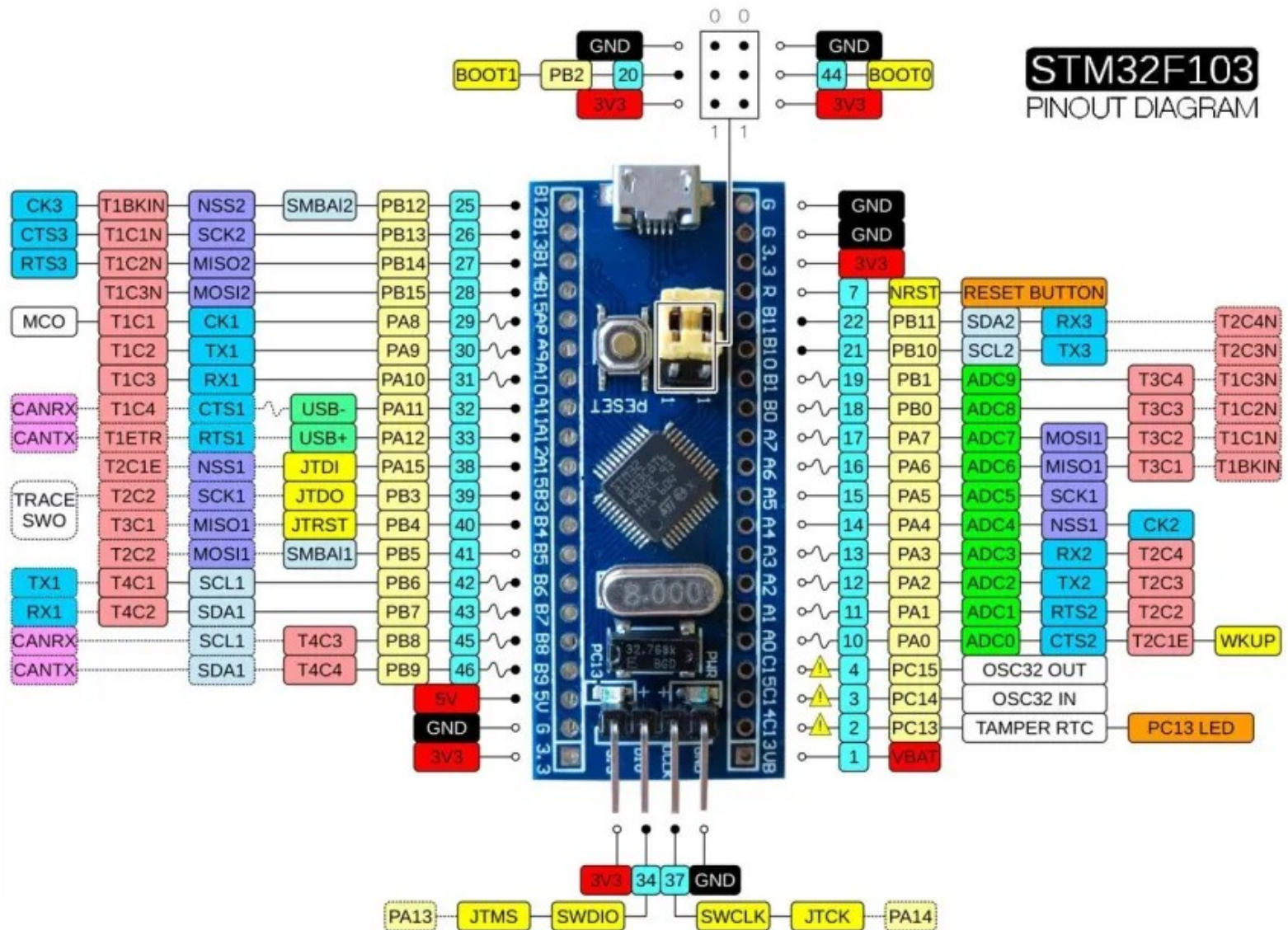
LEGEND

POWER
GROUND
PHYSICAL PIN
PIN NAME
CONTROL
ANALOG
TIMER & CHANNEL
USART
SPI
I2C
CAN BUS
USB
MISC
BOARD HARDWARE
● 5V tolerant
○ Not 5V tolerant
~ PWM pin
..... Alternate function
⚠ PC13,PC14,PC15: Sink max 3mA, source 0mA, max 2mhz, max 30pF

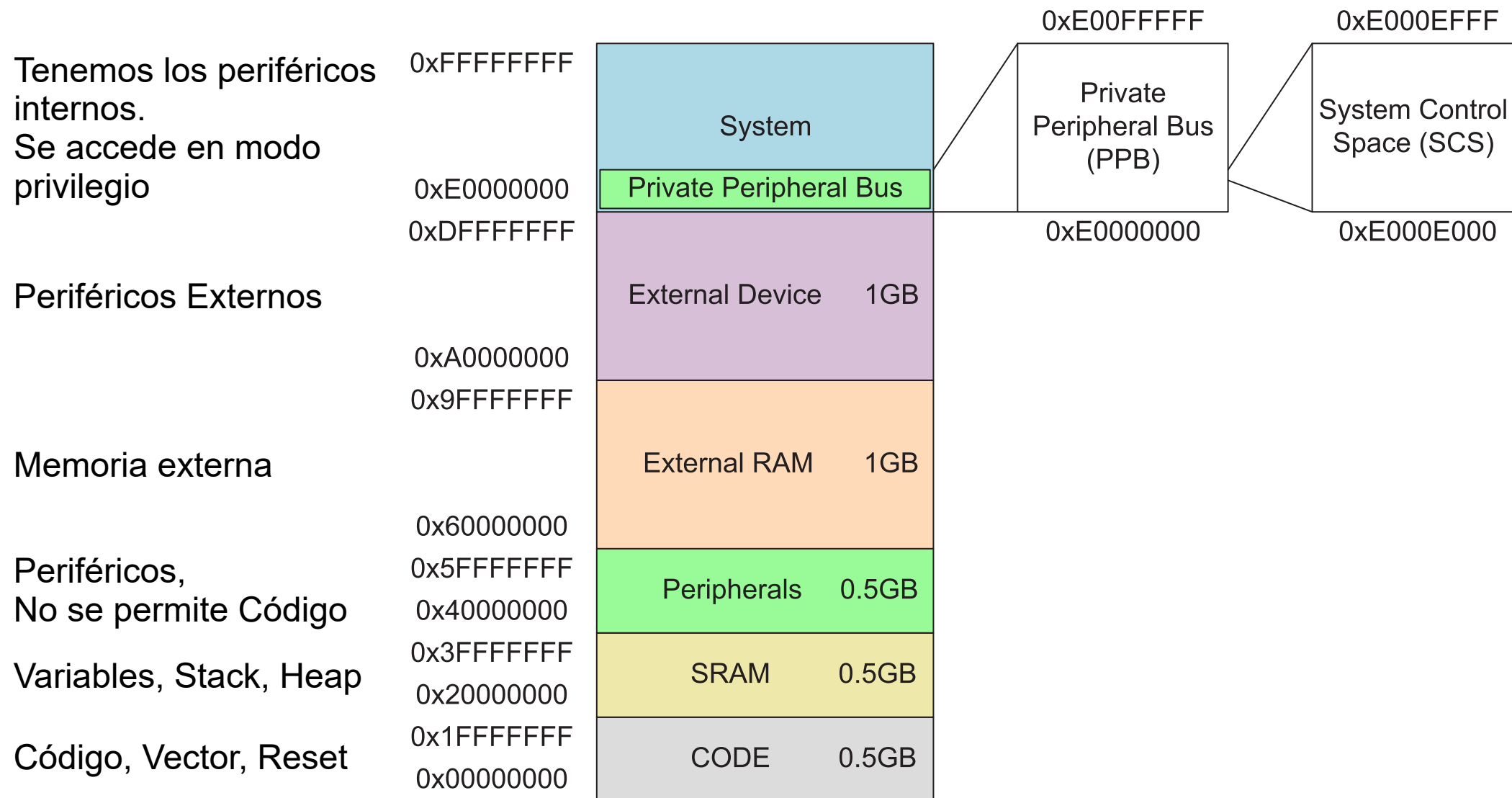
Absolute MAX 150mA
total source/sink for
entire CPU

Max ±20mA per pin,
±8mA recommended

STM32F103 PINOUT DIAGRAM



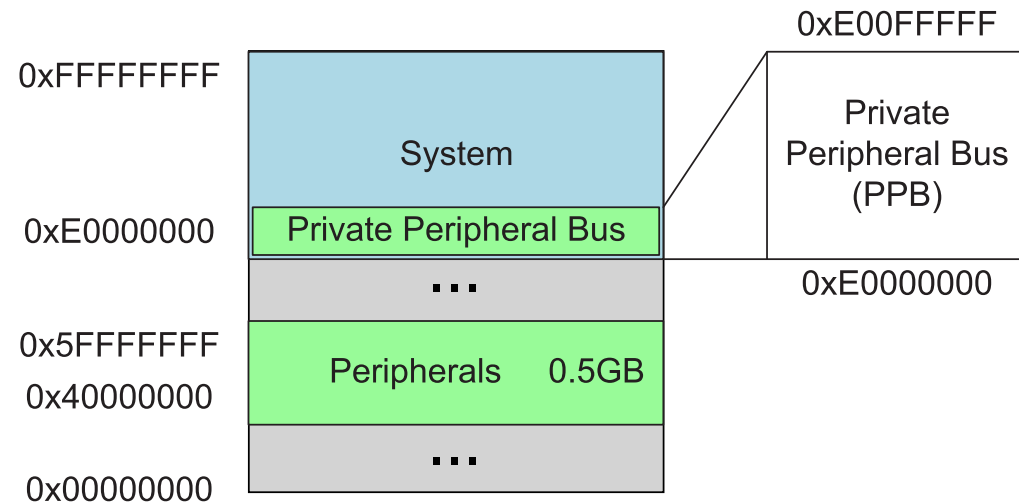
Periféricos en Cortex-M – Mapa General



Periféricos en Cortex-M

En la arquitectura ARM Cortex, no todos los periféricos son iguales. Se dividen en dos grandes grupos según su ubicación en el bus:

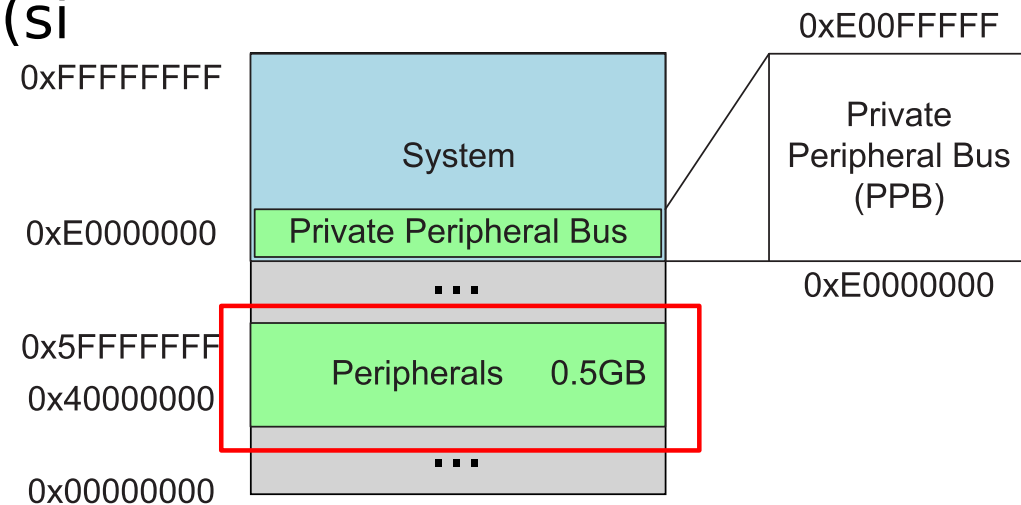
- **Periféricos Externos al núcleo** (del Fabricante): Diseñados por el fabricante, (STM para la Bluepill por ejemplo)
Son en general la GPIO, UART, ADC, Timers, etc.
A partir de 0x40000000
- **Periféricos Internos** (del Núcleo): Diseñados por ARM.
Son esenciales para el funcionamiento del procesador (NVIC, SysTick, SCB).
A partir de 0xE0000000



La Región de Periféricos Estándar (0x40000000)

Esta región es la que usamos para interactuar con el mundo físico.

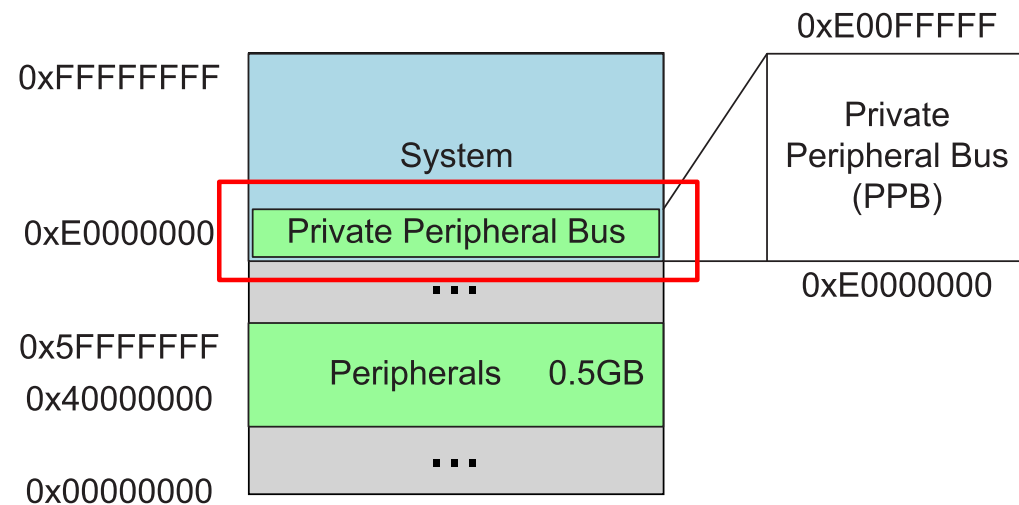
- Ubicación: 0x40000000 a 0x5FFFFFFF.
- Se accede a través del System Bus y se distribuye mediante puentes a los buses APB (lentos) y AHB (rápidos).
- Características:
 - Es donde configuramos los pines (GPIO) o los protocolos de comunicación.
 - Son específicos de la Blue Pill (si cambias de micro a un NXP, estas direcciones y registros cambian).



El Private Peripheral Bus (PPB) (0xE0000000)

El PPB es un bus dedicado exclusivamente a los componentes que hacen que el núcleo sea un "Cortex",
No depende del fabricante

- Ubicación: 0xE0000000 a 0xE00FFFFFFF.
- Es un bus de 32 bits interno. Al ser "privado", la CPU no tiene que competir con el DMA o el USB para por ejemplo configurar una interrupción.
- Componentes:
 - **SCS** (System Control Space): Configuración del procesador.
 - **NVIC**: Gestión de interrup.
 - **SysTick**: El reloj del sistema.
 - **MPU**: Protección de memoria.





¿Por qué se separan los buses?

- 1. Determinismo y Velocidad:** El acceso al NVIC debe ser instantáneo. Si estuviera en el mismo bus que una comunicación SPI pesada, el procesador podría tardar ciclos extra en atender una interrupción. El PPB garantiza latencia mínima.
- 2. Seguridad y Privilegios:** El PPB está protegido. Por defecto, solo se puede acceder en Modo Privilegiado. Esto evita que un error en el código de usuario realice un “reset” en el micro o deshabilite interrupciones críticas.
- 3. Portabilidad (Estandarización):** Al estar en el PPB, el NVIC siempre está en la misma dirección en cualquier chip ARM Cortex-M3. Esto permite que el mismo código de RTOS funcione en una Blue Pill o en un micro de Texas Instruments sin cambiar una sola línea del núcleo.



Finalmente vemos las 3 capas de un SE

Retomando lo que hablamos al principio sobre periféricos externo o internos al chip y sumando la división interna explicada, podemos hacer una distinción entre los 3 periféricos que encontramos en un SE

Microcontrolador

CPU

Core Peripherals (Internos al Núcleo): Están dentro del diseño de ARM (NVIC, SysTick). Se accede por el PPB.

On-Chip Peripherals (Externos al Núcleo, Internos al Chip): Son los que agrega ST u otro fabricante (GPIO, ADC, Timers). Están en el silicio, pero fuera del Cortex-M3. Se acceden por el System Bus.

Off-Chip / System Peripherals (Externos al Chip): Lo que está en la placa o el mundo real (Teclado, LCD, Sensores). Se conectan a los pines del chip por medio de los periféricos anteriores.



¿ Preguntas ?